when testing a utility function. To test that an exception *is* raised, you need to catch the exception, verify it is the right kind of exception, and fail if an exception wasn't raised. Alternatively, you can use a utility that does that for you.

The Python `unittest` module has a special assertion called `.assertRaises()`, which validates that an exception is raised within a code context block. If your code reaches the end of the block without raising an exception, the test case fails (by raising a different exception). The following listing shows code that adds an example function intended to raise a `ValueError` as well as a test to verify that it did so.

Listing 8.4 Testing an exception got caused

```
# bands/tests.py
...
def raises_an_error():
    raise ValueError()

class TestBands(TestCase):
    ...
    def test_raises_an_error(self):
        with self.assertRaises(ValueError):
            raises_an_error()
```

A sample function that raises ValueError

Asserts that a ValueError is raised before the context block exits

8.5 Authenticating and posting data

Let's tackle a view with a little more complexity. The `edit_venue()` view from chapter 7 creates or edits a `Venue` object and ensures only users associated with the `Venue` are allowed to make changes. To fully test this view, you need two users: a user that owns the `Venue` and a user that doesn't. As the view is responsible for both creating and editing the `Venue`, you don't need to create a `Venue` object to test with, as creating it will be part of the test. The structure of the view and the test are visualized in figure 8.5.

Recall that this view uses both GET and POST HTTP methods, with GET fetching the form and POST submitting it. You've already seen `.client.get()`; unsurprisingly, there is a corresponding `.client.post()`. This method takes a dictionary containing the form values that the post submits.

The first code you need to add for this scenario is to create test users. The following listing shows the creation of an owner and nonowner account in the `.setUp()` method. Django's `User` Model object has a special method on its query manager for creating users; this is done rather than just creating the object normally, as the password needs to be encrypted before it is stored in the database.

Listing 8.5 Add users for testing

```
# bands/tests.py
...
from django.contrib.auth.models import User
...
class TestBands(TestCase):
    def setUp(self):
```

Import Django's user model.